

# BÁO CÁO BTVN#3 — Agent đặt vé máy bay bằng LangChain

## 1. Mục tiêu

Xây dựng agent đặt vé máy bay khứ hồi một chặng **SGN → DAD**, sáng ngày **2026-10-07**, ngân sách  $\leq 2.000.000$  VND, theo kiến trúc: `sagent = goal + tools + loop + termination`

## 2. Goal

Trạng thái cần đạt:

Đặt được một vé **SGN → DAD** bay sáng **2026-10-07**, giá  $\leq 2.000.000$  VND, trạng thái **confirmed** và đã thanh toán.

## 3. Tools mockup (**flight\_tools.py**)

| Tool                                                   | Vai trò               | Observation chuẩn hóa                      |
|--------------------------------------------------------|-----------------------|--------------------------------------------|
| <code>search_flights(origin, destination, date)</code> | Liệt kê chuyến        | <code>{status: ok, flights: [...]}</code>  |
| <code>check_seat(flight_id)</code>                     | Giá + ghế trống       | <code>{status: ok, flight: {...}}</code>   |
| <code>book_seat(flight_id)</code>                      | Giữ chỗ               | <code>{status: held, code, booking}</code> |
| <code>pay(code)</code>                                 | Thanh toán            | <code>{status: paid, code, booking}</code> |
| <code>get_booking(code)</code>                         | Đọc lại để kiểm chứng | <code>{status: ok, booking}</code>         |

Mọi tool trả về `dict` có trường `status` tường minh: `ok` / `held` / `paid` / `invalid_param` / `not_found` / `sold_out` / `error`, kèm `hint` hướng phục hồi (sửa đúng 4 failure mode trong slide: sai định dạng ngày → gọi `YYYY-MM-DD`; kết quả rỗng → `{status: ok, flights: []}` thay vì `{}` mơ hồ). File này cũng expose các hàm `raw_*` để harness dùng trực tiếp và 5 tool `@tool` để cắm vào `create_agent`.

Dữ liệu mẫu (ngày 2026-10-07): VN122 08:10 / 1.850.000 (đáp án đúng), VJ604 09:30 / 2.080.000 (quá ngân sách), QH118 15:40 / 1.640.000 (sai giờ), VN134 10:20 / 1.950.000 (không hoàn → cần duyệt).

#### 4. Các lớp harness (`harness.py`)

| Lớp                                     | Cài đặt                                                                                                              |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Ràng buộc là dữ liệu                    | <code>BookingConstraints + satisfies()</code> / <code>progress_score()</code>                                        |
| Tiêu chí hoàn thành kiểm bằng code      | <code>check_completion()</code> — sensor computational, 0 token; cộng <code>cross_check_price()</code> chống bịa giá |
| Kiểm quyền (chạy <b>trước</b> thực thi) | <code>PermissionChecker.check()</code> — checklist #0                                                                |
| Phát hiện lặp / bế tắc                  | <code>LoopDetector</code>                                                                                            |
| Ngân sách (kiểm <b>cuối cùng</b> )      | <code>BudgetTracker</code> — kiểm sau cùng để không che mất chẩn đoán                                                |
| Bàn giao                                | <code>build_handoff()</code> + <code>TraceLogger</code> + <code>AgentResult</code>                                   |

Thứ tự checklist sau mỗi observation: hoàn thành → lặp → bế tắc → ngân sách.

#### 5. Ba mẫu thiết kế

- **`agent_react.py` — ReAct:** lập Suy luận → Hành động → Quan sát. Mỗi vòng chọn đúng 1 tool từ observation mới nhất; dừng khi model thôi gọi tool và `check_completion()` xác nhận bằng code. LangChain: `create_agent(..., middleware=[ModelCallLimitMiddleware(run_limit=12)])`.
- **`agent_plan_execute.py` — Plan-then-Execute:** một lần gọi model sinh trọn kế hoạch 5 bước (`make_plan`), duyệt plan, rồi thực thi tuần tự **không lập lại kế hoạch** — bước lỗi là abort toàn bộ. LangChain: thêm `HumanInTheLoopMiddleware(interrupt_on={book_seat, pay})`.
- **`agent_hybrid.py` — Lai (ReAct + Plan):** lập kế hoạch → thực thi k=2 bước → nếu observation đổi đáng kể (hết chỗ, đổi giá) thì lập lại kế hoạch. LangChain: `TodoListMiddleware + ModelCallLimitMiddleware`.

Ba file dùng chung `flight_tools.py` và `harness.py` (không copy-paste). Vì môi trường chấm bài chưa chắc có API key, mỗi file có chính sách quyết định dựa trên luật (rule-based, đóng vai model) để chạy offline tái hiện được; khi có `MODEL_NAME` (ví dụ `openai:gpt-4o-mini`) thì hàm `build*_langchain_agent()` cắm model thật vào cùng tools + harness.

## 6. Đánh giá hiệu quả 3 mẫu agent

### 6.1. Phương pháp

- Cùng một `BookingConstraints` (SGN → DAD, sáng 2026-10-07, ≤ 2M) và cùng 5 kịch bản cho cả 3 mẫu (`python evaluate.py`).
- Chỉ tiêu: `status` kết thúc `SUCCESS`, số bước tool-call, và tính trung thực khi thất bại (báo `LOOP` / `STALLED` / `NEEDS_APPROVAL` / `FAILED` kèm bản giao, tuyệt đối không bịa mã vé).

| Kịch bản                      | Ý nghĩa                                                                            |
|-------------------------------|------------------------------------------------------------------------------------|
| <code>standard</code>         | VN122 thỏa mọi ràng buộc                                                           |
| <code>dynamic_sold_out</code> | VN122 hết chỗ ngay sau khi search                                                  |
| <code>over_budget</code>      | Mọi chuyến sáng đều > 2M                                                           |
| <code>timeout_loop</code>     | <code>check_seat</code> luôn timeout                                               |
| <code>approval_only</code>    | Chỉ còn vé không hoàn (cần người duyệt; chạy với <code>auto_approve=False</code> ) |

### 6.2. Kết quả

| Kịch bản                      | ReAct      | Plan-then-Exec | Hybrid     |
|-------------------------------|------------|----------------|------------|
| <code>standard</code>         | SUCCESS(5) | SUCCESS(5)     | SUCCESS(4) |
| <code>dynamic_sold_out</code> | SUCCESS(5) | FAILED(2)      | SUCCESS(4) |
| <code>over_budget</code>      | FAILED(2)  | FAILED(5)      | FAILED(6)  |
| <code>timeout_loop</code>     | LOOP(3)    | FAILED(2)      | LOOP(5)    |

| Kịch bản      | ReAct             | Plan-then-Exec | Hybrid            |
|---------------|-------------------|----------------|-------------------|
| approval_only | NEEDS_APPROVAL(4) | FAILED(2)      | NEEDS_APPROVAL(3) |

Tỉ lệ thành công trên 2 kịch bản khả thi (**standard**, **dynamic**): ReAct 2/2, Hybrid 2/2, Plan 1/2.

### 6.3. Phân tích

- **standard**: cả 3 **SUCCESS**. Hybrid ít bước nhất (4) vì tìm được giá listing để chốt **VN122** sớm; ReAct/Plan tốn 5 bước vì verify thêm bằng **check\_seat** — đắt hơn một chút nhưng chắc chắn hơn khi giá listing lỗi thời.
- **dynamic\_sold\_out**: Plan **FAILED** ở bước 2 — lỗi bước đầu làm hỏng toàn bộ phía sau. ReAct thấy **sold\_out** ở V2 và chuyển sang **VN134** (V3–V5) nên **SUCCESS**; Hybrid **SUCCESS** nhờ lập lại kế hoạch sau observation đổi.
- **over\_budget**: cả 3 đều thất bại trung thực, không bịa vé **VN999**. ReAct dừng nhanh nhất (2 bước: search xong thấy không có ứng viên thỏa ràng buộc thì dừng + bàn giao); Plan chạy hết 5 bước mù quáng rồi mới rút kiểm chứng; Hybrid tốn nhất (6) vì cố lập lại kế hoạch bị loop.
- **timeout\_loop**: ReAct **LOOP** ở V3 và Hybrid **LOOP** ở V5 — bộ **LoopDetector** so (**tool**, **args**) bắt được việc gọi lại **check\_seat(VN122)** mà ngân sách vẫn còn, đúng demo 2 trong slide. Plan chỉ **FAILED** ở bước 2 (abort khi step lỗi, không phát hiện lặp).
- **approval\_only**: ReAct/Hybrid dừng **NEEDS\_APPROVAL** trước khi thực thi, kèm bàn giao (“đang ở đâu – định làm gì – vì sao hỏi”). Plan **FAILED** vì kế hoạch cứng ghi **VN122** (đã hết chỗ trong kịch bản) nên chưa bao giờ chạm tới cổng kiểm quyền — vừa cứng vừa mù rủi ro.

### 6.4. Kết luận và bảng chọn mẫu

| Mẫu               | Chọn khi                                | Rủi ro chính (kiểm chứng)                                             |
|-------------------|-----------------------------------------|-----------------------------------------------------------------------|
| ReAct             | Không đoán được số bước, môi trường đổi | Dễ lặp/trôi — đã chặn bằng <b>LoopDetector</b> + ràng buộc là dữ liệu |
| Plan-then-Execute | Cần duyệt plan trước, việc ổn định      | Kế hoạch lỗi thời — rút <b>dynamic_sold_out</b> , mù cổng duyệt       |

| Mẫu | Chọn khi                        | Rủi ro chính (kiểm chứng)                        |
|-----|---------------------------------|--------------------------------------------------|
| Lai | Việc dài + môi trường biến động | Debug khó hơn, tốn bước khi nhiệm vụ bất khả thi |